

火山引擎Mem0: 为 Agent 注入跨轮次、跨任务、跨时间的上下文记忆

季旭

字节跳动高级技术专家

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

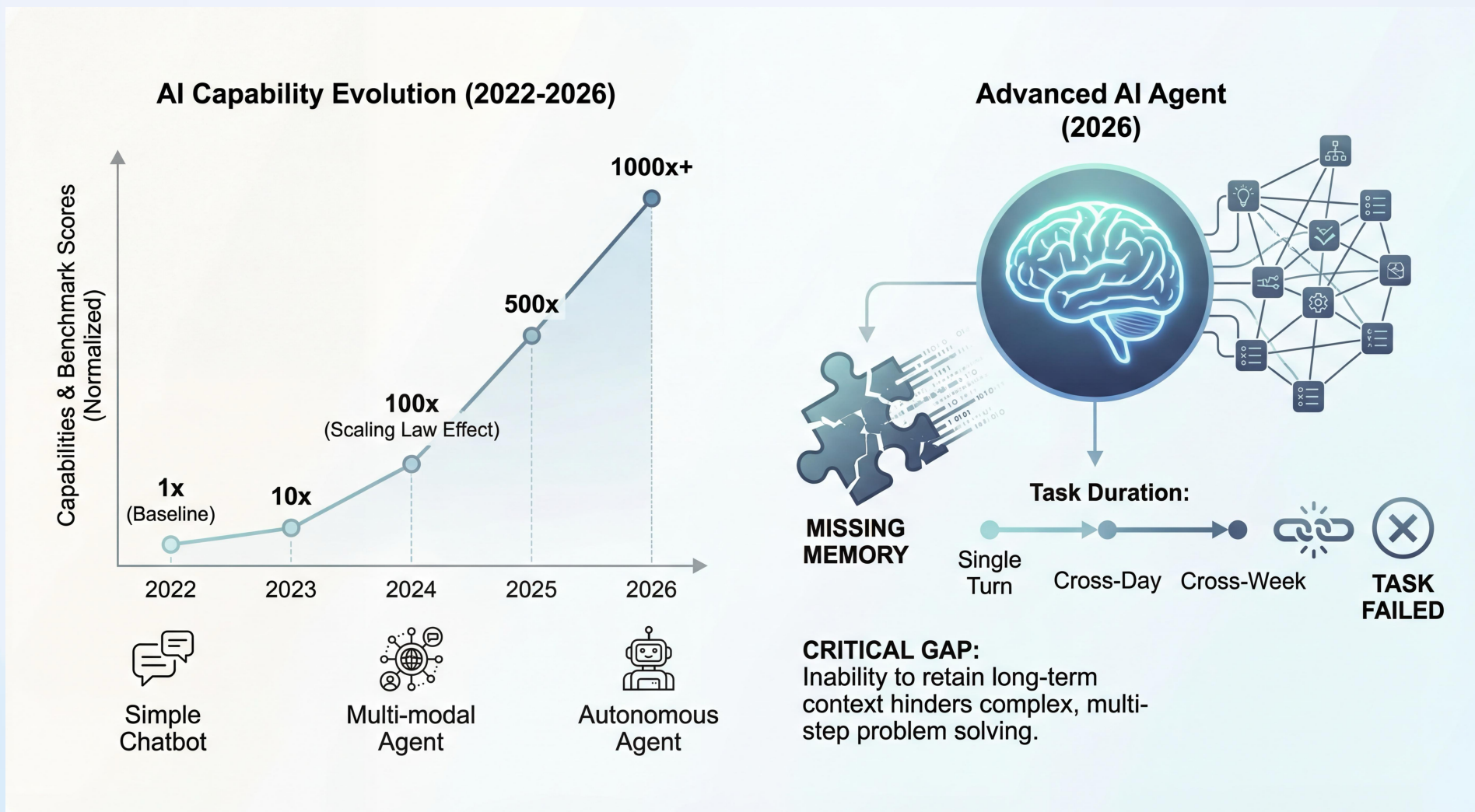
- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

目录

- 01 背景和痛点
- 02 火山引擎MemO 介绍
- 03 火山引擎MemO 核心优势
- 04 客户案例及总结

01 背景与痛点

Agent 时代的新挑战：记忆是缺失的关键拼图



痛点一：任务信息断裂

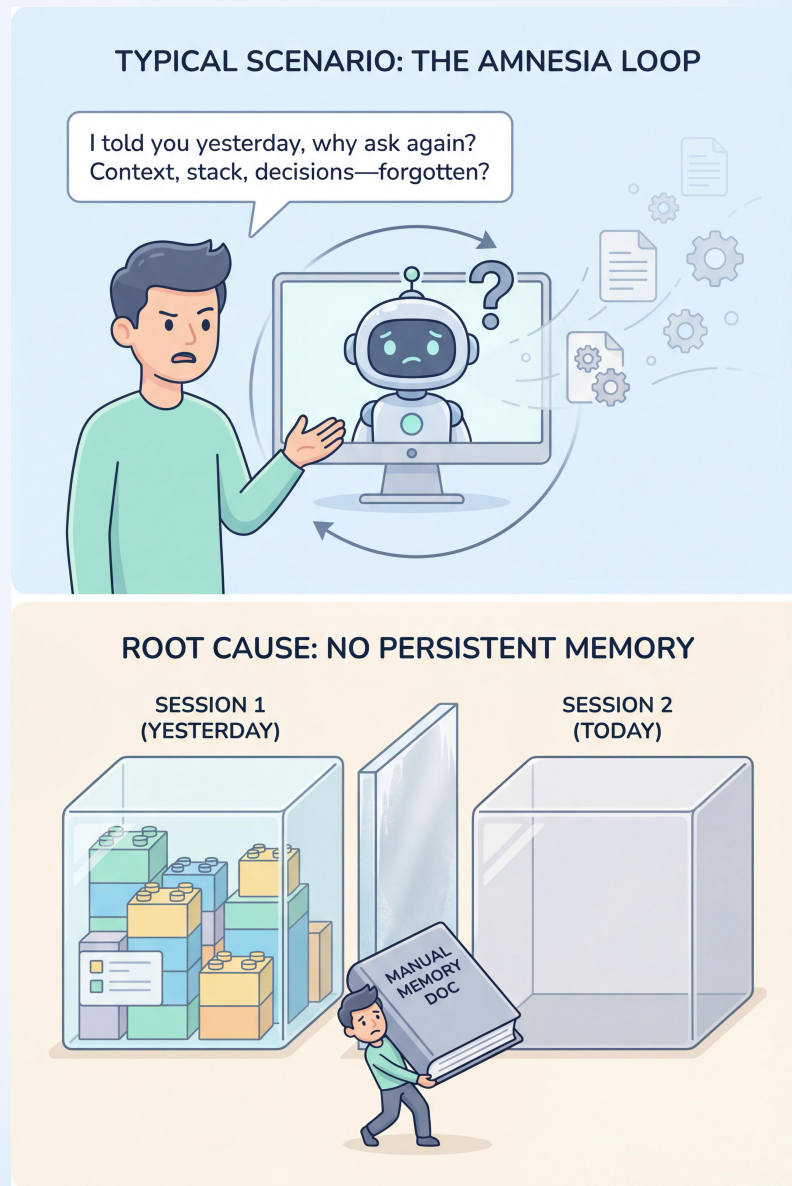
典型场景还原

一个项目刚刚推进完第一天的工作，第二天再让 Agent 继续，它却重新询问项目背景、重新阅读文档、重新梳理依赖关系，像第一次接手这个任务一样从零开始。

"我昨天已经跟你说过了，为什么又问我一遍？项目上下文、技术栈选型、已有的决策——你都忘了？"

根本原因

- Agent 无法在会话之间保留项目状态与上下文
- 每次新会话都从空白状态启动，历史信息全部归零
- 开发者必须手动维护"记忆文档"，增加额外负担



● 痛点二：经验无法沉淀

典型场景还原

线上故障的完整排查路径、关键监控指标、临时 workaround 和最终结论都散落在多个会话与沟通记录里。几周后类似问题再次出现，新的执行者仍然需要翻遍历史聊天、把排查流程完整地重走一遍。

"这个问题上次不是解决过了吗？当时怎么做来着？谁记得那个 workaround？"

根本原因与业务损耗

- 经验从**未被结构化沉淀**为可复用的上下文资产
- 知识只**存在于离散的聊天记录**中，无法检索
- **团队重复踩坑**，MTTR（平均故障恢复时间）居高不下
- 高价值的排查**经验随着人员流动彻底消失**

每次"重新排查"背后，都是工程师数小时的重复劳动与潜在的业务风险窗口。



● 痛点三：用户偏好反复丢失

典型场景还原

用户已经在多次对话中明确了自己的偏好与边界：输出风格要简洁、合规要求不能包含外部链接、语言习惯偏向技术术语、隐私约束需要脱敏处理。但一旦会话拉长、任务切换或上下文窗口超限，Agent 仍可能遗忘这些要求。

"我不是说了不要用 emoji 吗？怎么又加了？还有，我说了不要引用外部链接！"

根本原因

- 偏好信息仅靠 System Prompt 或会话上文维持，无持久化机制
- 上下文窗口压缩时偏好被丢弃，优先级低于任务内容
- 无法跨任务、跨会话持续生效，用户体验严重受损



根本原因：偏好信息仅靠 System Prompt 或会话上文维持，无持久化机制



上下文窗口压缩时偏好被丢弃，优先级低于任务内容



无法跨任务、跨会话持续生效，用户体验严重受损



● 痛点四：Token 成本爆炸

问题根源

为了避免 Agent 遗忘关键信息，开发者往往将大量历史对话、背景文档直接塞入提示词。这种"暴力上下文"策略会迅速推高 token 消耗、拉长响应延迟，并将大量与当前任务无关的噪声带入推理链路。

"为什么一次对话就花了这么多钱？能不能只给它真正需要的信息？"

成本估算

- 若每天 100 次对话 × 每次 10K token 历史上下文
- 单天额外消耗：**100 万 token**
- 额外成本（Opus4.6 计价）：约 **\$10 - \$100**
- 实际场景中并发量往往更高，成本呈指数级增长



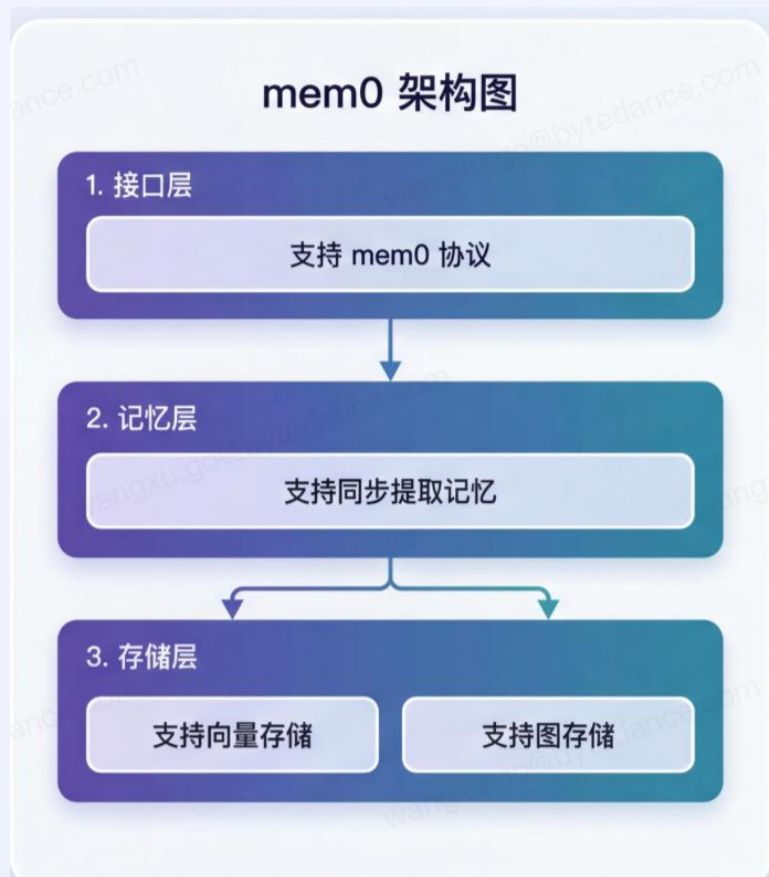
痛点总结：Agent 的记忆缺陷

五大痛点的共同根源不是模型缺乏单次理解能力，而是 Agent 缺少一套持续生效的上下文记忆系统。

缺陷维度	具体表现	业务影响
上下文窗口限制	长任务遗忘早期关键约束与决策	决策质量下降，输出前后矛盾
无状态与重复劳动	不同会话反复介绍项目背景	效率严重损耗，用户体验差
个性化难	偏好、风格、合规边界难以持续生效	用户满意度低，信任度下降
可靠性与可追溯性不足	没有全量交互记录与审计轨迹	质量无法闭环，问题难以定位
成本与延迟高企	记忆塞进 Prompt 导致 Token 膨胀	经济性差，响应速度慢

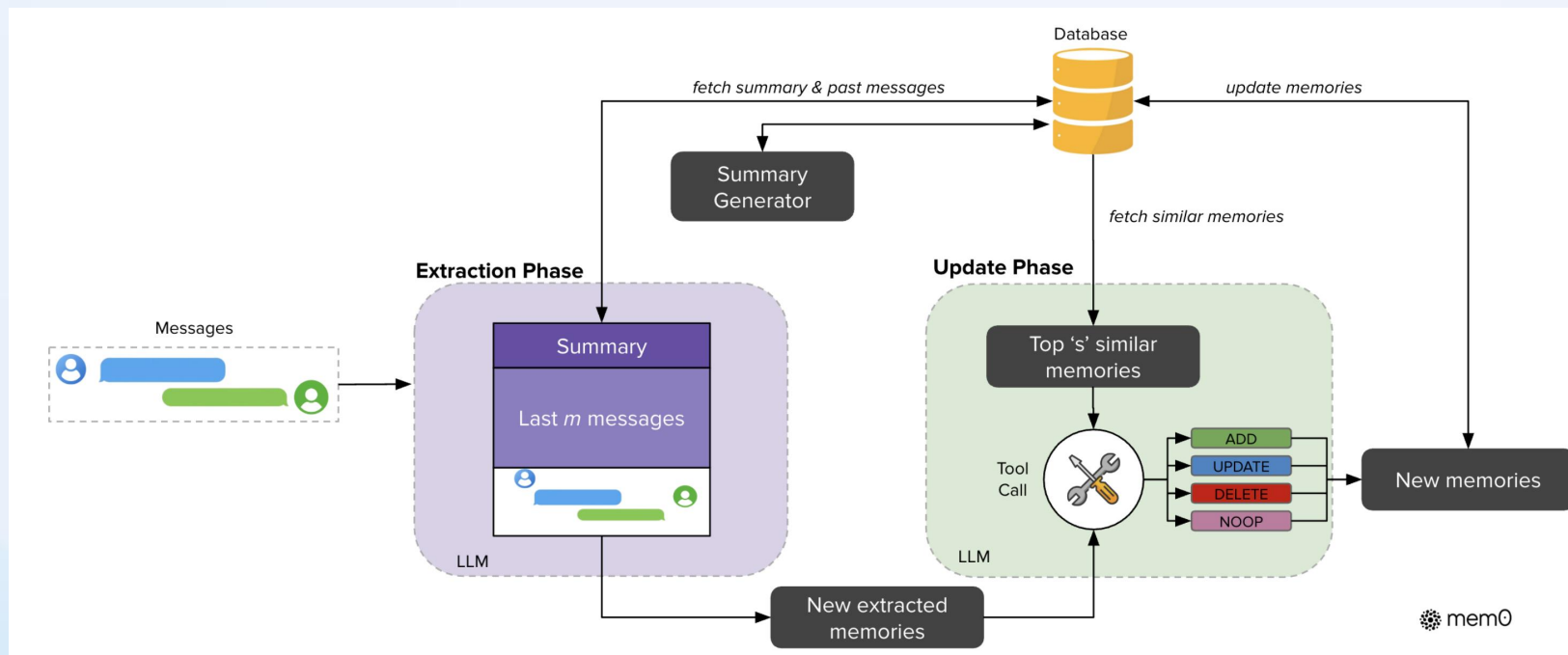
核心结论：这些问题的解决方案不在于更大的上下文窗口，而在于一套智能的、分层的、可持续进化的记忆管理系统。

社区解决方案: Mem0



Mem0 记忆提取/召回

通过LLM来提取记忆，而非传统RAG的方式来保存原文





MemO 的局限

记忆管理粗粒度

仅支持简单的 key-value 式记忆存储，缺乏层次化数据组织（标题 → 摘要 → 细节）

有限图谱能力

只能处理简单实体关系推理和单跳问答场景

无自进化机制：

记忆只能被动存取，无法从执行轨迹中自动提取经验并持续优化

企业级能力缺失：

缺乏多租户隔离、审计追溯、权限管控等企业级特性

召回策略单一

仅支持向量相似度召回，无法融合列表查询和图谱查询

02

火山引擎Mem0介绍

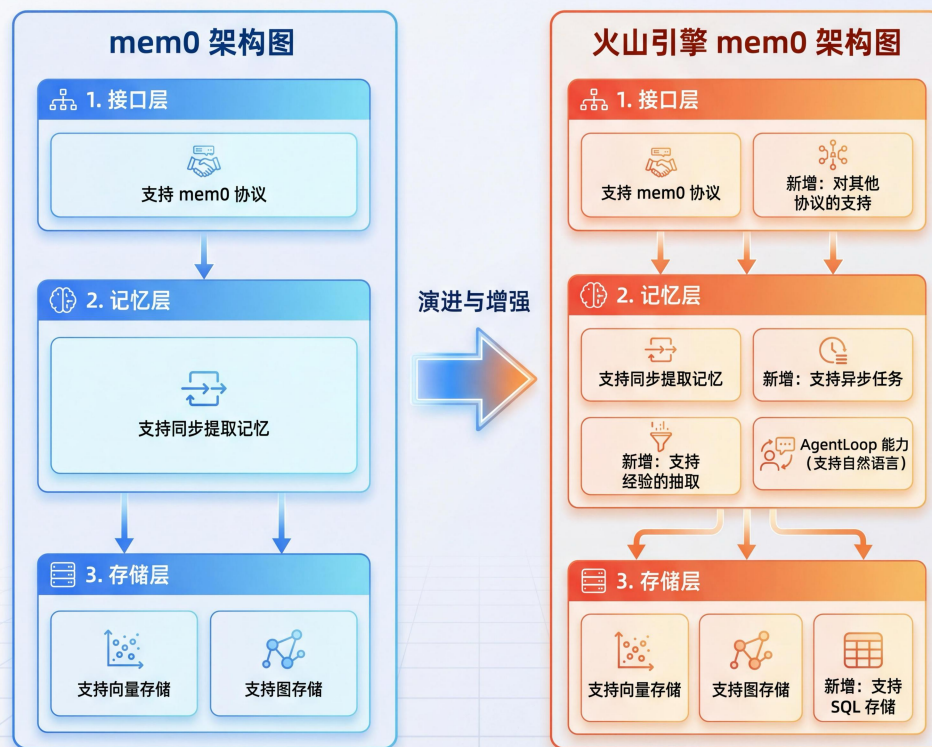
从 Mem0 到 火山引擎Mem0

火山引擎Mem0 面向企业级 Agent 平台的**完整记忆系统重构**，在六个核心维度实现全面升级。

对比维度	Mem0 开源版	火山引擎Mem0
数据组织	扁平 Memory 列表，无层次结构	三层模型：原始记忆 → Memory → Graph，树图结构
检索方式	单一向量相似度检索/较弱的图谱能力	列表 / 向量 / 图谱三路融合排序召回
图谱能力	基础 / 实验性支持	自动图谱构建，多跳推理，实体关系追踪
自进化	无	记忆自进化：经验提取 → 评估 → 进化 → 调度
企业特性	无权限管理、无审计、无多租户	完整审计日志、多租户隔离
协议兼容	Python SDK	RESTful API + MCP 协议 + Mem0 API 兼容层

火山引擎Mem0 架构

架构对比：mem0 与 火山引擎 mem0



记忆数据组织

三层数据模型设计理念

火山引擎Mem0 将大模型 Context 按层次化结构组织成由多棵树构成的图，实现从原始数据到结构化知识的渐进式抽象。

Layer 1: History (原始记录)

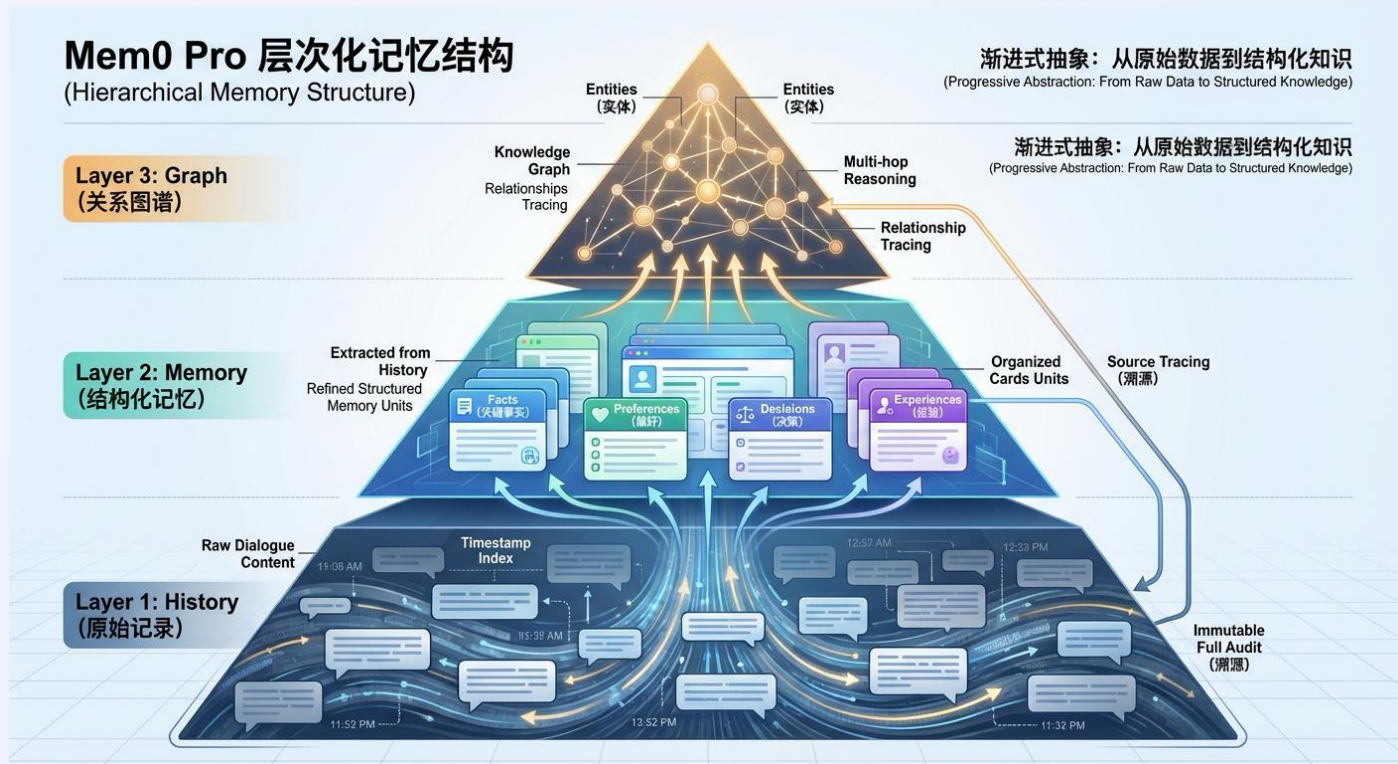
完整保存每轮交互的原始对话内容，支持全量审计与回溯。时间戳索引，不可篡改。

Layer 2: Memory (提炼记忆)

LLM 从 History 中抽取关键事实、偏好、决策与经验，形成精炼的结构化记忆单元。

Layer 3: Graph (关系图谱)

从 Memory 中自动识别实体与关系，构建知识图谱，支持多跳推理与关系追溯。



树图结构使得 Agent 既能快速检索精炼记忆，又能在需要时溯源到原始交互记录，兼顾效率与可靠性。

火山引擎Mem0 三层查询

短期记忆

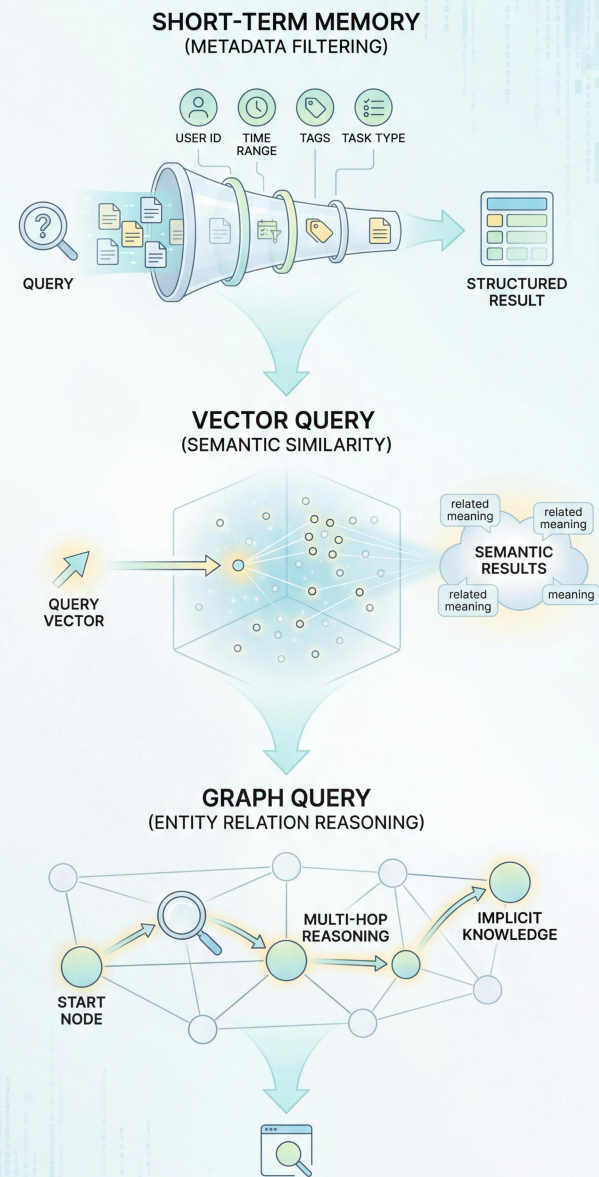
基于元数据过滤：用户 ID、时间范围、标签分类、任务类型。精确匹配，延迟极低，适合已知条件的结构化检索。

向量查询

基于语义相似度的嵌入向量检索，找到语义相关的 Memory 单元。即使措辞不同，也能召回语义相近的历史记忆。

图谱查询

基于实体关系的图遍历查询，支持多跳推理。可以找到不直接相关但通过关系链接的隐含知识。



03

火山引擎Mem0核心优势

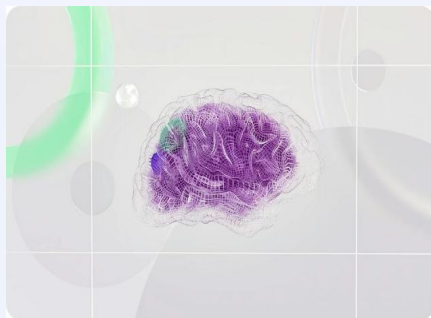
火山引擎Mem0 核心优势

火山引擎Mem0 的差异化竞争力体现在三个可量化的核心价值：**精准记忆注入节省 Token**、**记忆自进化驱动能力提升**、**Graph Memory 增强复杂推理**。



精准记忆注入

Token 消耗降低 **92.8%**，正确率提升 **104.3%**



记忆自进化

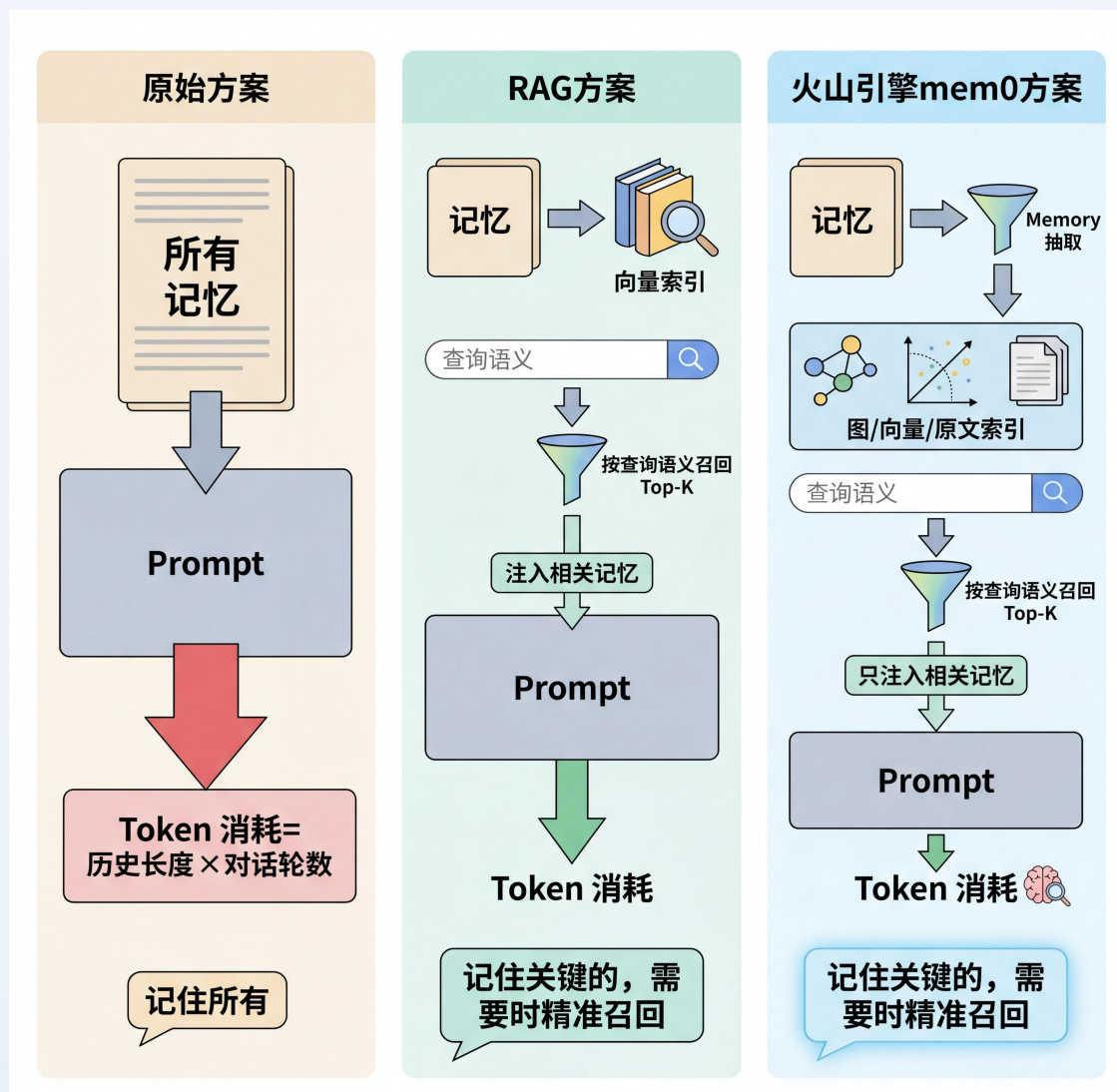
从失败案例中自动提取经验，持续迭代，冷启动成本大幅降低



Graph Memory

自动构建实体关系图谱，支持多跳推理与因果链路追溯

优势一：精准记忆注入



Evaluation: LoCoMo benchmark

- 定位：面向极长多轮对话的记忆评测基准，专门测试 Memory 系统的效果
- 数据规模：300+ 轮对话、9000+ Token 原始数据、1940 个测试问题

评测维度	场景 / 任务类型	核心评估目标	示例
长期对话记忆	跨多个 Session 的超长双人对话	在涉及数周甚至数月的多次对话（Multi-session）中，回答关于用户偏好、过往经历或特定事实的问题。	例如：结合我上周提到的度假目的地和我昨天说的预算
问答与推理	基于长对话历史的问答（QA）	重点看模型能不能在一大段历史对话里 精准找到相关信息 ，并处理好跳转题、多跳推理题、时间顺序题、常识题以及带一点“刁钻”设计的对抗性问题。	例如：问“列出过去两周我尝试修复电脑的完整时间线及每一步的结果。”，模型需要把多次对话里的信息拼在一起才能回答完整。
事件级理解	要求模型对整个对话历史进行结构化理解，识别事件的演进过程。	因果与时序的一致性。考察模型能否在长对话中 抓住关键事件 ，把时间线理清楚，构建出合理的 时间顺序和因果链 。	例如：根据一段情侣从争吵到和好的长对话，总结出完整故事线：先是谁迟到、然后引发争吵、接着谁道歉、最后一起去看电影和好。
多模态对话	文本 + 图像混合的多模态对话生成	希望模型在利用历史文本和图像信息的基础上，生成既 符合角色设定、上下文连贯 ，又能真正看懂图里关键信息的回复。	例如：你发了一张家里客厅的照片问“这张照片里适合再加一盏什么样的灯？”，模型需要结合图片里的装修风格以及你之前聊到的喜好来给出建议。

LoCoMo Benchmark 评测结果

在 LoCoMo 基准评测中，火山引擎Mem0 在 Token 消耗和回答正确率两个核心维度上均取得显著领先。

评测系统	Token 消耗（相对）	正确率	备注
OpenClaw + memory-core2	51,018,371	32.5%	全量历史塞入方案
OpenClaw + LanceDB	51,086,935	41.5%	纯向量检索方案
OpenClaw + 火山引擎Mem0	1,097,220 (↓92.8%)	66.4% (↑104.3%)	记忆抽取 + 精准注入

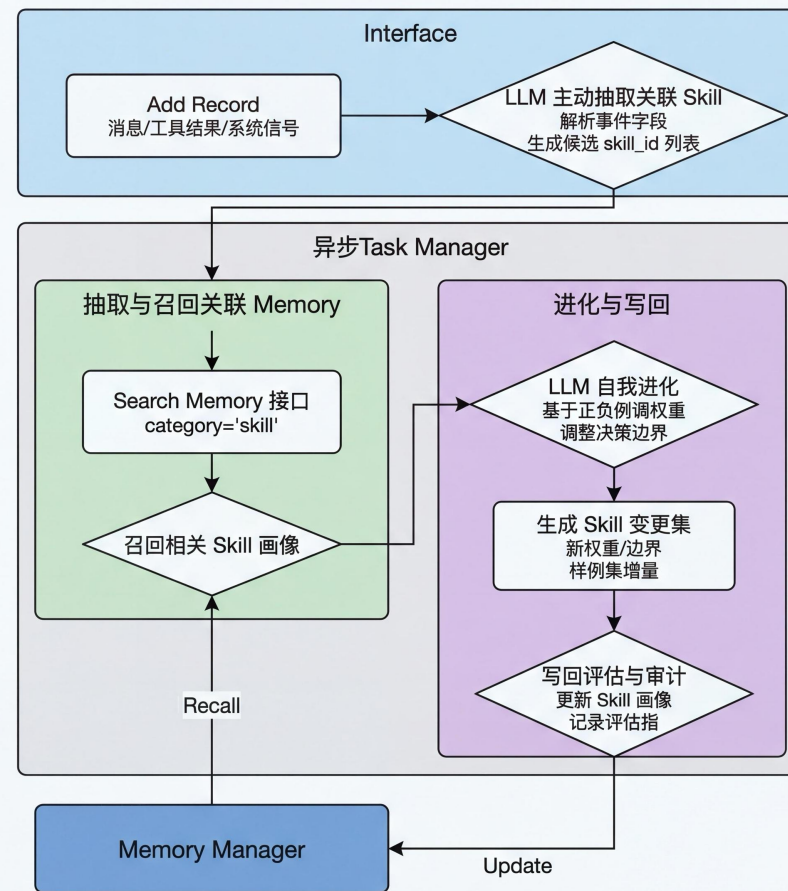
得益于火山引擎Mem0的记忆抽取能力，相比于原生的OpenClaw，正确率提升104.3%，token消耗降低92.8%

优势二：记忆自进化

核心理念： Agent 不仅能“记住”，还能从运行轨迹中“学习”，将经验提炼为可复用的经验或Skill

核心流程：

- add record: 记录 Agent 完整执行轨迹（输入、工具调用、中间结果、最终输出）
- LLM 抽取关联经验：析轨迹中的成功模式和失败教训
- search_memory 召回相关经验：检查是否已有类似经验
- 记忆自进化：通过正向样例（成功）和负向样例（失败）迭代优化经验内容



OfficeQA

定位：面向真实企业办公场景的 Agent 能力评测，由DataBricks开发

场景覆盖：邮件处理、会议纪要总结、文档分析、表格操作、IM 消息理解

评测维度：多源异构数据 + 长上下文 + 多步推理 + Agent 工具调用能力

与传统 QA benchmark（如 SQuAD、Natural Questions）相比，OfficeQA 更贴近真实工作流，具有以下特点：

多源异构数据：涵盖邮件（Email）、会议纪要（Meeting Notes）、文档（Docs）、表格（Sheets）、即时通讯（Chat）等

长上下文依赖：问题往往依赖跨文档、跨时间的信息整合

任务导向：不仅是“问答”，还包含总结、决策支持、流程追踪等

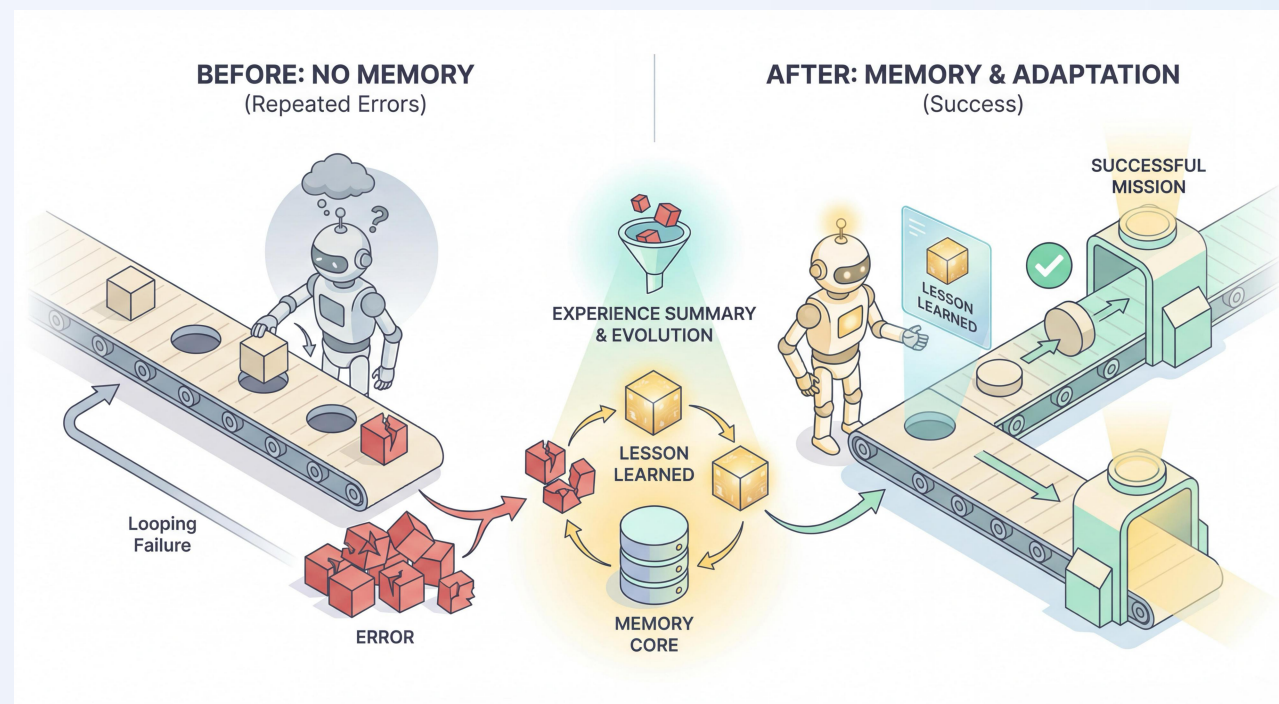
多步推理：需要 multi-hop reasoning，而非单文档抽取

OfficeQA 的**核心目标**是评估模型在企业知识工作（knowledge work）场景中的实际可用性，尤其适用于评测结合 RAG、memory（如 mem0）、以及工具调用能力的系统。



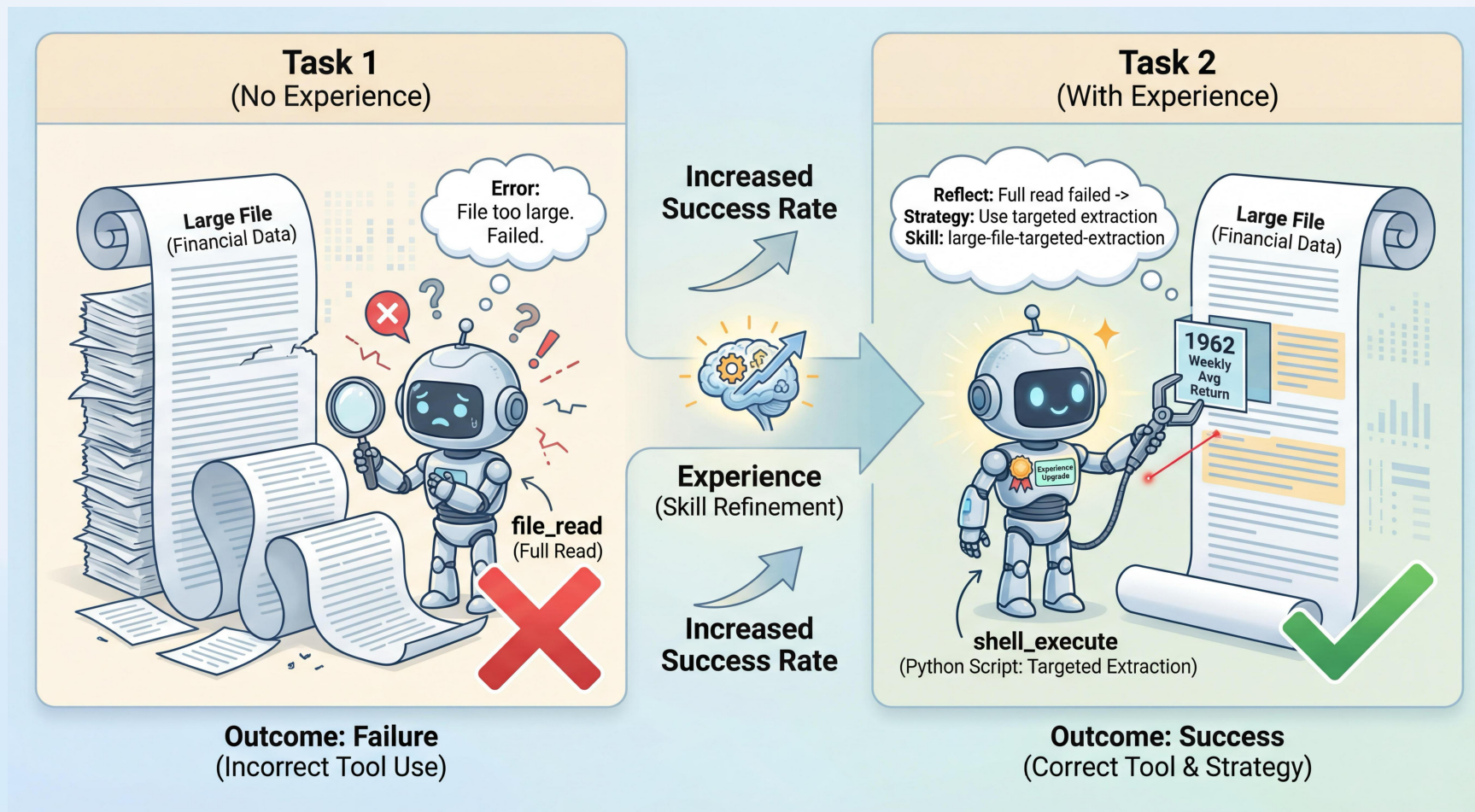
实验结果：自进化前后对比

方法	OfficeQA准确率
openclaw+built-in memory	39.0%
openclaw+mem0	42.3%
openclaw+火山引擎 Mem0	52.3%(13.3%)



实验分析

通过失败的例子学习到了读超大文件不应该使用file_read，总结了大文件读取的相关经验

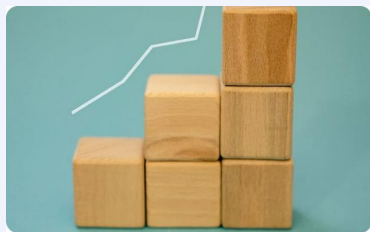


记忆自进化的价值



降低成本

新 Agent 部署后无需从零积累经验，可直接继承已有经验库。原本需要数周磨合期的任务，可在数天内达到高质量输出水平，大幅缩短从部署到产值的周期。



持续提升质量

每次任务执行都是一次学习机会。成功路径强化经验，失败路径触发反思与修正，形成正向飞轮。Agent 质量随时间单调递增，而非停滞不前。



可控可审计

所有经验的来源、版本历史与演化路径均有完整记录。管理员可审查、回滚或手动干预经验内容，确保自进化过程在可控范围内进行，满足企业合规要求。

优势三：Graph memory

维度	传统Memory	Graph Memory
数据组织	文本分块（chunk）存储，基于语义或词法检索	通过模型/规则，将文本转换为图结构（如知识图谱或社区层次），基于实体、关系和图遍历检索
检索逻辑	直接匹配查询与文本块的相似性	通过实体匹配、图遍历或多跳推理检索相关子图或社区
优势场景	单跳问题、细节信息检索	多跳推理、全局关联分析、复杂关系推理
局限性	难以捕捉跨文本块的隐含关系	图构建可能不完整，全局检索可能丢失细节
示意图		

Graph Memory擅长场景



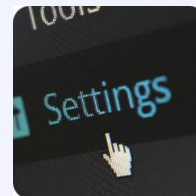
人脉关系推理

场景："这个项目的最终审批人是谁？" 图谱遍历：项目 → 负责人 → 汇报关系 → 审批链路，自动找到决策节点，无需用户逐步提示，极大提升跨组织协作效率。



因果链路追溯

场景："上次线上故障的根本原因是什么，影响了哪些下游系统？" 图谱记录了故障事件与各系统间的依赖关系，可以自动重建完整的因果传播路径，支持根因分析与影响评估。



偏好继承与传播

场景：用户 A 设定了输出风格偏好，其管理的子项目应继承相同规范。图谱通过层级关系自动传播偏好约束，无需每个子项目单独配置，确保风格一致性与合规性。

Graph Memory实验结果

直接使用Chatbot的进行验证结果

系统配置	Topk 召回	输入Token	Locomo 准确率
火山引擎Mem0 (graph-only)	30	2983.2448 * 1540	89.1%(28% up)
mem0	30	3817.3446 * 1540	67.4%
mem0g	30	5066.2034 * 1540	67.9%

04

客户案例及总结

客户案例

飞书妙虾



工程挑战:

- 快速迭代: OpenClaw快速迭代, 记忆范式在不断进化
- 海量新实例: 大量新用户注册, 每个用户的数据都需要在记忆侧隔离
- 活跃实例比例较低: 闲时需要节省资源, 降低成本

火山引擎Mem0 核心价值

价值	核心能力	关键结果
节省 Token	精准记忆注入替代全量历史	Token 消耗 ↓ 92.8% ，正确率 ↑ 104.3%
记忆自进化	从执行轨迹自动抽取经验，持续迭代优化	OfficeQA 验证集正确率随训练轮次持续提升 13.3%
Graph	知识图谱 + 向量检索融合	正确率提升 28%

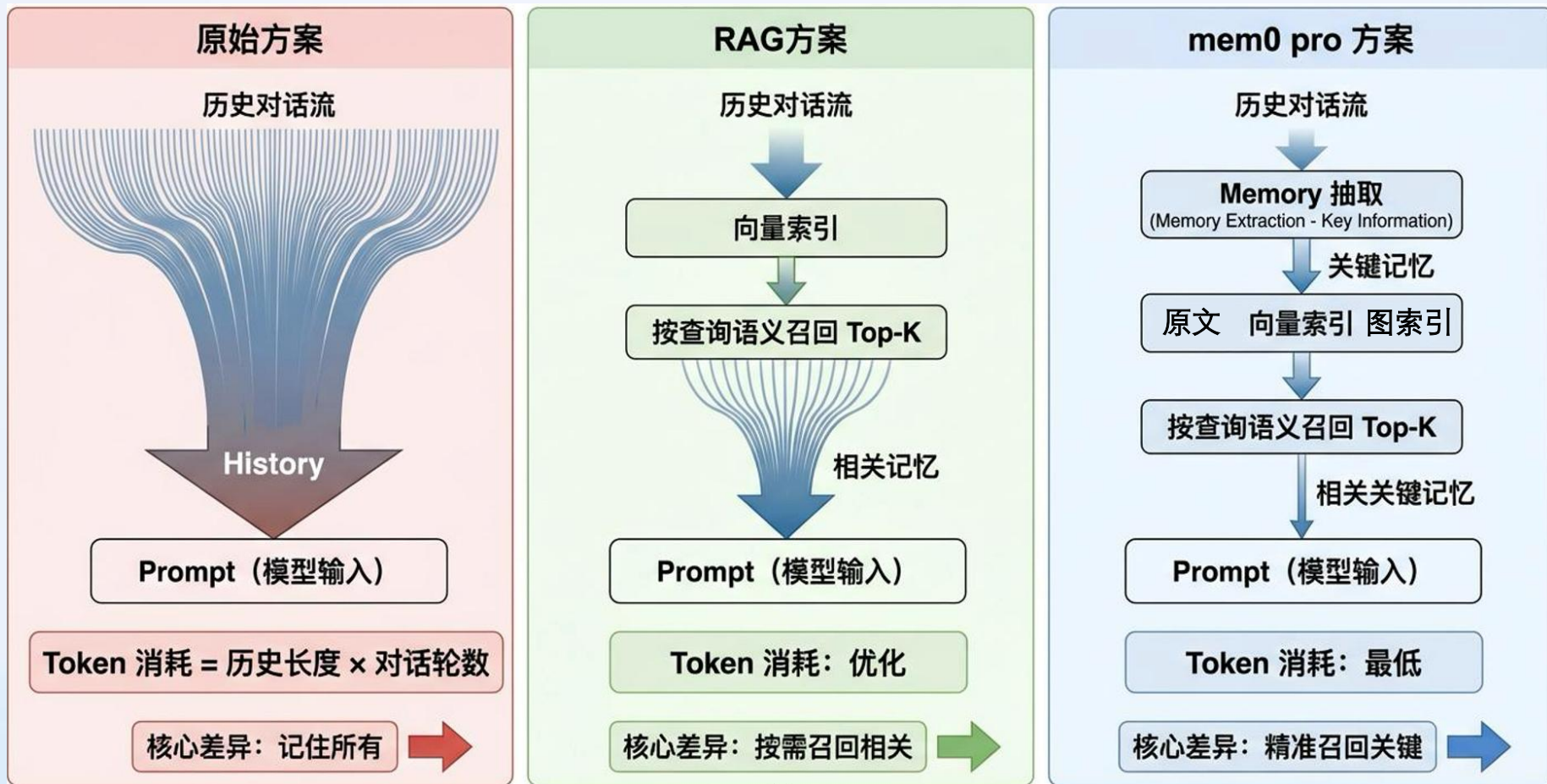


THANKS

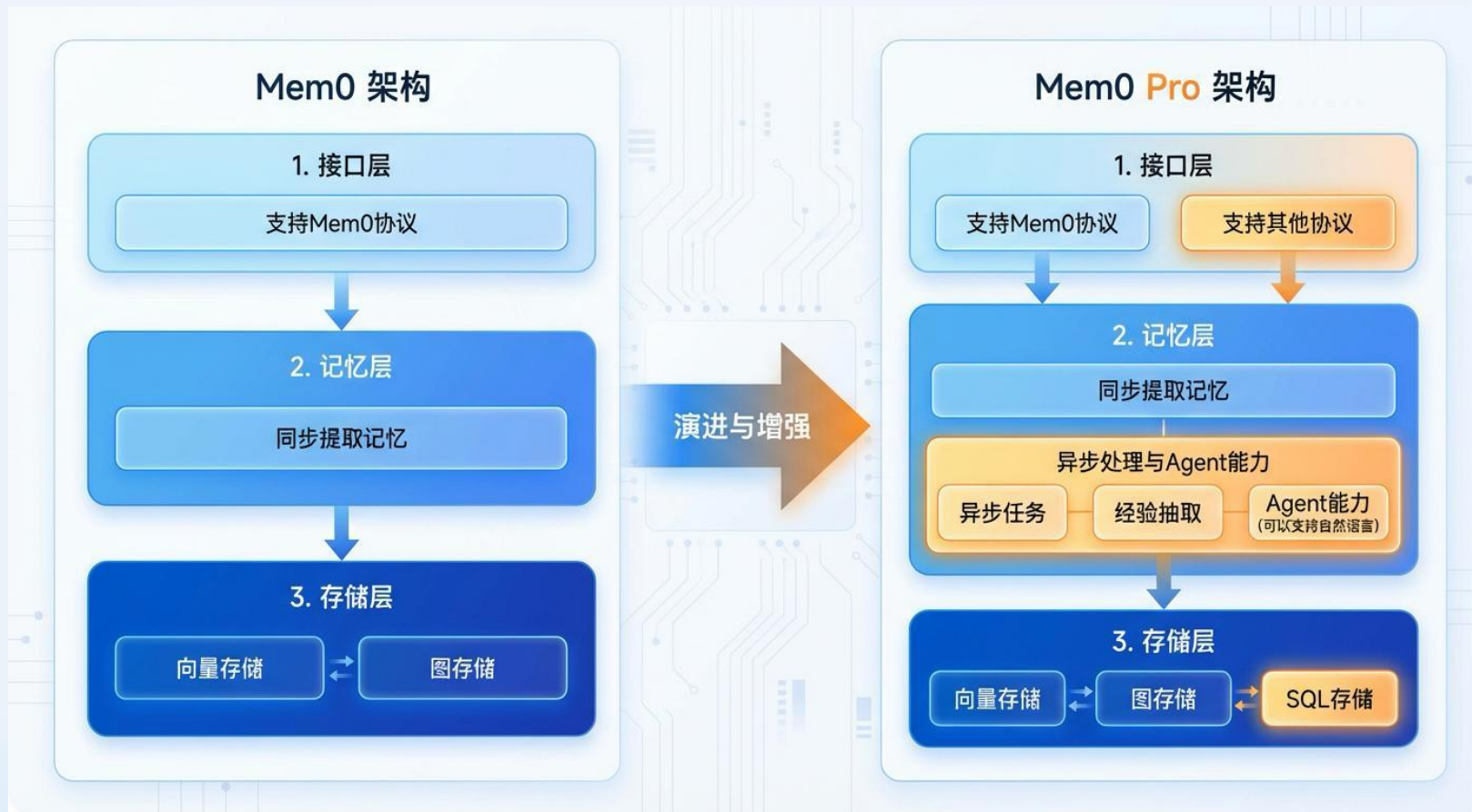
火山引擎Mem0: 为 Agent 注入跨轮次、跨任务、跨时间的上下文记忆



优势一：精准记忆注入



火山引擎Mem0 架构



社区解决方案: Mem0



AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

